

Cloud Computing und Big Data

Prüfungsleistung 2026 - Datengetriebener Prototyp auf Kubernetes-Basis

1. Aufgabenstellung

In Gruppenarbeit ist ein prototypischer, datengetriebener Big-Data-Dienst für einen selbst gewählten Use Case zu entwickeln und deklarativ auf Kubernetes zu betreiben. Bewertet wird nicht Produktionsreife, sondern ob die Idee verstanden und in einer kohärenten Architektur umgesetzt ist. Ein lauffähiger Prototyp mit nachvollziehbarem Datenfluss genügt.

- Gruppengröße: nach Vorgabe der Veranstaltung (eine Abgabe pro Gruppe)
- Architektur: Kappa (streaming-first) als Standard. Eine andere Architektur (z. B. Lambda) ist zulässig, muss aber im Bericht begründet werden.
- Technologien frei wählbar, sollten aber zum Lehrstoff passen (z. B. Kafka, HDFS, Spark Structured Streaming, Data Lake / Lakehouse mit Delta/Parquet als Storage, beliebige Serving-/Query-Schicht).
- Abweichungen von den in der Vorlesung gezeigten Systemen sind ausdrücklich erlaubt und erwünscht (z. B. Objektspeicher wie MinIO/S3 statt HDFS, eine andere Streaming-Engine, ein anderes Tabellenformat). Gut begründete Abweichungen werden mit Bonuspunkten honoriert (siehe Bewertungsschema).

Anforderungen an den Prototyp

Die Anwendung muss alle Stufen einer Streaming-Datenpipeline abbilden:

- Ingestion: Aufnahme eines (synthetischen oder echten) Datenstroms
- Stream Processing: mindestens eine nicht-triviale Transformation (z. B. Aggregation über ein Zeitfenster, Join, Stateful Processing, Anreicherung)
- Storage: Persistenz der Ergebnisse in einem Data Lake / Lakehouse (Format, Partitionierung und Schema müssen begründet sein)
- Serving: Bereitstellung der Ergebnisse (API, Query-Schicht oder Visualisierung)

Zusätzlich verpflichtend ist eine User-Facing UI:

- Eine Weboberfläche zur Interaktion mit dem System
- Mindestens in der Rolle des Datenlieferanten (Erzeugen/Einspeisen von Events in die Ingestion) und/oder zur Anzeige der verarbeiteten Ergebnisse
- Die UI muss real an die Pipeline angebunden sein (kein Mockup), als eigene Komponente containerisiert und auf Kubernetes deployt

Betrieb auf Kubernetes:

- Alle Komponenten containerisiert und über Manifeste oder ein Helm-Chart deklarativ deployt
- Sinnvolle Workload-Typen (Deployment/StatefulSet), Konfiguration über ConfigMaps/Secrets, persistente Daten über PVCs
- Ein dokumentierter, reproduzierbarer Deploy-Weg (Helm, Kustomize, ...)
- Skalierbarkeit: die Anwendung muss darauf ausgelegt sein, in allen Komponenten horizontal zu skalieren und dies soll gezeigt werden.

2. Abgabeformat

Die Bewertung erfolgt anhand der Abgabe bestehend aus README.md, Code (alle Sourcen, Deployment-Dateien, Manifeste, Daten, etc.), und Screenshot (ohne dass die Anwendung ausgeführt wird). Der Ordneraufbau des Codes ist frei, muss aber aus der README heraus verlinkt sein. Die Abgabe enthält die vorgenannten Komponenten in einer einzigen ZIP-Datei. Nur diese wird bewertet. Es werden keine externen Dateien heruntergeladen oder angeschaut.

README.md (alleiniges Berichtsdocument)

Die README.md (Markdown-Format) enthält alle Inhalte. Die Abschnitte entsprechen den Bewertungskriterien; fehlende Pflichtabschnitte werden im jeweiligen Kriterium mit 0 Punkten gewertet.

1. Use Case und Motivation: Problem, Datenquelle, warum Big-Data-Problem
2. Datencharakteristik: relevante V's (Volume/Velocity/Variety)
3. Architekturentscheidung: Kappa vs. Lambda mit Begründung, plus Architekturdiagramm
4. Komponenten und Datenfluss: jede Komponente, Technologiewahl begründet, Ende-zu-Ende-Fluss
5. Processing-Logik: Transformationen, Windowing/State, Late Data
6. Speicherkonzept: Format, Partitionierung, Schema, warum Data Lake/Lakehouse
7. User-facing UI: Rolle (Datenlieferant/Anzeige), Anbindung an die Pipeline, Bedienablauf
8. Kubernetes-Deployment: Abbildung der Komponenten auf Workloads, Config, Persistenz, Skalierung
9. Deployment-Anleitung: reproduzierbarer Weg, Voraussetzungen
10. Wesentliche Codeabschnitte: Verlinkung auf die zentralen Dateien/Zeilen im Repo (Ingestion, Processing, UI, Manifeste), je mit einem Satz, was dort passiert

11. Screenshots und Nachweise: eingebettete Screenshots des laufenden Systems (UI im Betrieb, kubectl get pods, Serving-Output) und Beispiel-Outputs der Pipeline
12. Grenzen des Prototyps und Ausblick — ehrliche Einordnung des Scopes

Da die Anwendung nicht ausgeführt wird, ersetzen die eingebetteten Screenshots und Beispiel-Outputs den Funktionstest. Die Bilddateien liegen in einem Ordner und werden in der README eingebunden.

3. Bewertungsschema

Insgesamt max. 100 Punkte.

Kriterium	Punkte	Volle Punktzahl bedeutet
Use Case und Big-Data-Begründung	10	Klares Problem, passende Datenquelle, überzeugende Begründung warum Big Data (V's konkret)
Architektur und Begründung (Kappa/Lambda)	20	Saubere Architekturwahl mit Begründung, korrektes Diagramm, Komponenten konsistent zum Datenfluss
Processing-Logik und Verständnis	20	Nicht-triviale Transformation, Code implementiert sie nachvollziehbar, Windowing/State/Late-Data adressiert
Speicherkonzept	10	Format/Partitionierung/Schema begründet, passend zu Data Lake/Lakehouse
User-facing UI	10	Real an die Pipeline angebundene UI (Datenlieferant/Anzeige), als Komponente deployt, Bedienablauf nachvollziehbar
Kubernetes-Deployment	15	Korrekte, kohärente Manifeste/Helm, sinnvolle Workload-Typen, Config über ConfigMap/Secret, PVCs, deklarativ und Skalierbarkeit
Reproduzierbarkeit und Struktur	10	Vollständige, klar gegliederte README, eindeutiger Deploy-Weg, verlinkte Codeabschnitte, eingebettete Screenshots belegen den Lauffähigkeit
Reflexion und Eigenanteil	5	Ehrliche Scope-Grenzen, klare Aufgabenverteilung, plausible Git-History

Bonus: Eigenständigkeit und Innovation (bis +10)

Zusätzlich zu den 100 Kernpunkten können bis zu 10 Bonuspunkte vergeben werden für nachvollziehbar begründete Eigenständigkeit, die über den gelehrten Standardweg hinausgeht:

- Begründete Abweichung von den in der Vorlesung gezeigten Systemen (z. B. Objektspeicher statt HDFS, alternative Streaming-Engine oder Tabellenformat)
- Besonders origineller oder anspruchsvoller Use Case
- Funktionalität, die über den geforderten Mindestumfang hinausgeht (z. B. Exactly-once-Semantik, Schema-Evolution, sinnvolle Autoskalierung, CI/CD)

Voraussetzung ist immer eine Begründung im Bericht. Abweichung ohne Begründung gibt keinen Bonus. Reine Mehrarbeit ohne erkennbaren Mehrwert ebenfalls nicht. Die Maximalpunktzahl ist und bleibt auch mit Bonuspunkten 100.